WELT 0 · DOPPELSTUNDE 1 · KICK-OFF · TEIL B: NOTATIONS-REFERENZ FÜR DS 2

W0·1 – Das Portal: Start ins Dungeon und Notation fürs Abitur

🎯 Im Lab: **Level 0.1 „Das Portal: Notation fürs Abitur“** · Online-IDE: Level 0 „Diagnose“ · Datei: `AQ_W0_L0_Diagnose_Geruest.txt`

Willkommen im Game-Studio! In den nächsten 14 Wochen baut euer Kurs ein Dungeon-Spiel – und dabei genau die Algorithmen und Datenstrukturen, die im Abitur drankommen: Suchen und Sortieren, Rekursion, Stapel, Schlangen und Bäume. Sieben Welten liegen vor euch, am Ende jeder Welt wartet ein Boss-Kampf.

Doch bevor euer Studio Helden schmieden darf, durchläuft es die **Akademie der Baumeister** – und zuerst steht ihr vor ihrem **Portal**. Es lässt nur Teams durch, die ihr Werkzeug beherrschen: das **Algo Quest Lab** zum Üben, die **Online-IDE** zum Programmieren und Testen – und **Papier und Stift**. Denn im Abitur gibt es keinen Compiler. Java-Methoden, Struktogramme und Tracetabellen schreibst du dort von Hand. Darum trainieren wir beides vom ersten Tag an.

Teil A – Start: Team, Lab, Werkzeug, Diagnose

Infoteil: So funktioniert Algo Quest

- Ihr arbeitet in **Teams** (Tischgruppen mit 3–4 Personen): Ihr erklärt euch gegenseitig und prüft eure Lösungen im Code-Review – aber jede Person schreibt ihren eigenen Code.
- Geübt wird im **Algo Quest Lab**: Auf der **Weltkarte** hat jede Welt Lab-Level mit interaktiven Aufgaben, die sofort automatisch geprüft werden. Dazu kommen ein **Trainingsplatz** mit Zufallsaufgaben, ein **Online-Boss** je Welt und der **Warm-up-Trainer**.
- **Punkte** gibt es im Lab automatisch beim ersten Lösen – niemand muss etwas eintragen. Dein **Rang** steigt automatisch mit: Novize → Lehrling → Geselle → Meister → Großmeister („abiturbereit“). Dazu sammelst du **Abzeichen**.
- **Punkte zählen nicht für die Note.** Das Scoreboard im Lab zeigt Namen und Punkte nur innerhalb eures Kurses.
- Jede Aufgabe ist gekennzeichnet: [IDE] = am Gerät, [PAPIER] = ohne Gerät, nur mit Stift. Die Sterne zeigen die Stufe: ★ Basis, ★★ Erweiterung, ★★★ Experte.
- Jede Doppelstunde hat denselben Rhythmus: 10 min Papier-Warm-up (ohne Gerät) · 15 min Input · 50 min Lab-Level und Online-IDE · 15 min Sicherung.

Wo?	Was?	Punkte
Algo Quest Lab	Lab-Level: Aufgaben der Stufe 1 · 2 · 3 – Abzug für Fehlversuche und Tipps, „Lösung zeigen“ = 0	**10 · 20 · 30** automatisch
Algo Quest Lab	Trainingsplatz (Zufallsaufgaben) · Online-Boss am Ende jeder Welt	**1 je Aufgabe · bester Versuch zählt**
Online-IDE	IDE-Level Basis · Erweiterung · Experte, Papier-Compiler	ohne Punkte – die Tests zeigen, was läuft
Papier	Papier-Warm-up, Papier-Aufgaben, Papier-Boss	ohne Punkte – Training fürs Abitur

Aufgabe 1: Euer Team und eure Regeln

PAPIER ★

a) Setzt euch in eurem **Team** zusammen (Tischgruppe, Einteilung durch die Lehrkraft). Stellt euch kurz vor: Was konntet ihr in der Einführungsphase gut, was weniger?

b) Lest die Team-Regeln im Kasten. Formuliert gemeinsam **eine zusätzliche Regel** für euer Team und notiert sie unten.

Unser Team (Vornamen):	
Unsere Zusatzregel:	

Team-Regeln

1. Ihr helft euch gegenseitig – aber jede Person schreibt **ihren eigenen** Code und **ihre eigenen** Papier-Lösungen.
2. Wer hilft, erklärt statt vorzusagen – als Helfer:in, ohne Punkte, aber mit dem besten Lerneffekt.
3. Tests in der IDE werden nie verändert. Wer glaubt, dass ein Test oder eine Lab-Aufgabe falsch ist, meldet ihn (Bug-Meldung – ohne Punkte).
4. Punkte zählen nicht für die Note. „Lösung zeigen“ im Lab ist erlaubt – aber erst, wenn ihr es wirklich versucht habt.
5. Dein persönlicher Lab-Code gehört nur dir. Du notierst ihn im Heft – nicht auf diesem Blatt.

Aufgabe 2: Das Portal öffnet sich

IDE ★

a) **Beitritt ins Algo Quest Lab.** Öffne algoquest.frankjuchim.de oder scanne den QR-Code am Beamer. Wähle „Kurs beitreten“ und gib den **Kurscode** und deinen **Namen** ein – Vorname + Initial des Nachnamens, z. B. „Lena S.“. Das Lab zeigt dir deinen **persönlichen Code** (z. B. K7M-42Q): **Notiere ihn sofort in deinem Heft!** Mit Kurscode + Name + Code spielst du auf jedem Gerät weiter („Weiterspielen“).

b) **Online-IDE einrichten.** Öffne online-ide.de und melde dich mit deinen Zugangsdaten an. Lege einen neuen Workspace **AlgoQuest_W0** an (oder öffne den Workspace, den die Lehrkraft verteilt hat). Erstelle darin eine Datei und füge den Inhalt von `AQ_W0_L0_Diagnose_Geruest.txt` ein.

c) **Testrunner ausprobieren.** Öffne den Reiter **„Testrunner“** und starte die Testklasse `DiagnoseBasisTest` mit dem grünen Dreieckspfeil. Notiere: Wie viele Tests sind rot, wie viele schon grün? Warum sind manche Tests schon grün, obwohl du noch nichts programmiert hast?

Rundgang im Lab

- **Weltkarte:** Welt 0 „Die Akademie“ mit Level 0.1 bis 0.6, dem Trainingsplatz und dem Online-Boss „Aufnahmeprüfung: Die Bibliothek der Akademie“.
- **Level-Seite:** Merkkasten, Aufgabenliste und die Karte „Programmieren in der Online-IDE“ – sie nennt das passende IDE-Level (ohne Punkte).
- **Aufgabe:** „Prüfen“, bei Bedarf „Tipp“, im Notfall „Lösung zeigen“ (danach 0 Punkte). Nach dem Lösen erscheint eine Erklärung.
- **Profil:** Punkte, Rang, Abzeichen, „Deine Fallen“ (P1–P10) und dein Code. Außerdem: **Scoreboard** und **Warm-up-Trainer**.

Ausblick: Bibliothek „niedersachsen“

Ab Welt 4 brauchst du die Klassen `Stack`, `Queue`, `DynArray` und `BinTree` aus dem Abitur. Sie stecken in der Bibliothek **„niedersachsen“**, die man pro Workspace einschaltet (Rechtsklick auf den Workspace → Einstellungen). **Jetzt noch nicht nötig.**

Vermerk: Lehrkraft prüft den Menüpfad vor dem Einsatz.

Aufgabe 3: Level 0 – Die Diagnose

IDE ★ – ★★★

Das Portal prüft, was du aus der Einführungsphase noch kannst: Reihungen, Schleifen, Verzweigungen und Zeichenketten. Keine Panik – das ist eine Diagnose, keine Klassenarbeit. Jeder grüne Test zeigt dir, was schon sitzt. Du arbeitest nur in der Klasse `Diagnose` im Abschnitt „DEIN TEIL“.

Stufe	Testklasse	Methoden (Signatur wie im Abitur)
★ Basis	DiagnoseBasisTest	summe(r: Reihung von Ganzzahl): Ganzzahl maximum(r: Reihung von Ganzzahl): Ganzzahl (r ist nicht leer) zaehle(r: Reihung von Ganzzahl, x: Ganzzahl): Ganzzahl
★★ Erweiterung	DiagnoseErweiterungTest	indexVon(r: Reihung von Ganzzahl, x: Ganzzahl): Ganzzahl (erstes Vorkommen, sonst −1) istAufsteigendSortiert(r: Reihung von Ganzzahl): Wahrheitswert durchschnitt(r: Reihung von Ganzzahl): Kommazahl (leere Reihung → 0.0)
★★★ Experte	DiagnoseExperteTest	anzahlVokale(s: Zeichenkette): Ganzzahl (a, e, i, o, u – klein und groß) istPalindrom(s: Zeichenkette): Wahrheitswert (exakter Zeichenvergleich)

a) Arbeite die Stufen der Reihe nach ab. Eine Stufe ist geschafft, wenn **alle** Tests ihrer Testklasse grün sind.

b) Lies bei roten Tests die Meldung – sie verrät dir, welcher Fall noch nicht klappt. Trage zwei Fehler, die du gemacht hast, in dein **Fehlerlogbuch** ein (Tabelle unten; im Heft auf den letzten Seiten weiterführen).

c) Halte dich an die Spielregeln des Abiturs: Zeichenketten nur mit den erlaubten Operationen bearbeiten (Teil B) – also **kein** `toLowerCase`. Reihungen laufen von Index 0 bis Länge − 1.

Tipp: Mit dem grünen Startknopf läuft das Hauptprogramm ganz unten in der Datei. Dort kannst du deine Methoden mit eigenen Werten ausprobieren.

Mein Fehlerlogbuch

Nr.	Was war falsch?	Warum? (Ursache)	So vermeide ich es
1			
2			
3			

Teil B – Notation fürs Abitur (Referenz für DS 2)

Im Abitur gibt es keinen Compiler und keinen Testrunner. Wer die Notation aus den Ergänzenden Hinweisen (EH 2025) sicher beherrscht, verschenkt keine BE. Die folgenden Infoteile gibt es auch als laminierte Werkzeugkarten.

Infoteil 1: Signaturen und Typnamen

Methoden werden im Abitur sprachunabhängig angegeben: **name(parameter: Typ, …): Rückgabetyp**

Beispiel: zaehle(r: Reihung von Ganzzahl, x: Ganzzahl): Ganzzahl ⟷ `public static int zaehle(int[] r, int x)`

Typname im Abitur	Java	Typname im Abitur	Java
Ganzzahl	`int`	nichts (keine Rückgabe)	`void`
Kommazahl	`double`	Reihung von Ganzzahl	`int[]`
Wahrheitswert	`boolean`	Reihung von Gegenstand	`Gegenstand[]`
Zeichen	`char`	Klassenname, z. B. Held	`Held`
Zeichenkette	`String`		

- Es heißt immer **Reihung**, nie „Array“. Der Index beginnt bei 0 und endet bei Länge − 1. Bei 2D-Reihungen: `r[zeile][spalte]`.
- Die Länge einer Reihung auslesen (`r.length`) ist erlaubt, die for-each-Schleife auch.
- `/` rechnet bei Ganzzahlen ganzzahlig: 7 / 2 = 3. `%` liefert den Rest: 7 % 2 = 1.

Infoteil 2: Struktogramm-Elemente (EH § 5)

Struktogramme sind sprachunabhängig. Zuweisungen schreibst du mit ←, Vergleiche mit =, ≠, <, ≤, >, ≥. Bedingungen verknüpfst du mit **und**, **oder**, **nicht**.

Element	Struktogramm-Notation	Java

Kopf mit Signatur	**summe(r: Reihung von Ganzzahl): Ganzzahl**
	… *// Anweisungen der Methode*
		`public static int summe(int[] r) {`
		`  ...`
		`}`

Zuweisung	anzahl ← anzahl + 1	`anzahl = anzahl + 1;`
Objekt erzeugen (ruft den Konstruktor auf)	h ← erzeuge Held("Aria")	`Held h = new Held("Aria");`
Methodenaufruf	h.nimmSchaden(30)	`h.nimmSchaden(30);`
Rückgabe – beendet die Methode sofort	gib summe zurück	`return summe;`

Verzweigung (Dreieck-Kopf mit Bedingung, links wahr, rechts falsch)	r[i] > max
	wahr / falsch
	max ← r[i]
		`if (r[i] > max) {`
		`  max = r[i];`
		`}`

Solange-Schleife (kopfgesteuert)	solange summe < ziel
	…
		`while (summe < ziel) {`
		`  ...`
		`}`

Zählschleife – „bis“ ist einschließlich!	für i von 0 bis Länge von r − 1
	…
		`for (int i = 0; i < r.length; i++) {`
		`  ...`
		`}`

Element	Struktogramm-Notation	Java

Für-jedes-Schleife	für jedes x in r
	…
		`for (int x : r) {`
		`  ...`
		`}`

Kommentar	hilf ← r[i] *// Wert retten*	`hilf = r[i];  // Wert retten`

Infoteil 3: Die Tracetabelle

- Eine **Spalte je Variable** – bei Bedarf zusätzlich eine Spalte für Rückgabe oder Ausgabe. Bedingungen bekommen keine eigene Spalte.
- Eine **neue Zeile** an jeder Stelle, die im Code mit „// → Tabellenzeile“ markiert ist (meist vor der Schleife und am Ende jedes Durchlaufs). Unveränderte Werte darfst du weglassen.
- Gibt die Aufgabe die Spalten vor, hältst du dich genau daran.

Beispiel: Aufruf `summe(r)` mit r = {4, 7, 1}

```
int summe = 0;   // → Tabellenzeile (Startwerte)
for (int i = 0; i < r.length; i++) {
    summe = summe + r[i];   // → Tabellenzeile
}
return summe;
```

Schritt	i	r[i]	summe
1			0
2	0	4	4
3	1	7	11
4	2	1	12
Ende			Rückgabe 12

Infoteil 4: Erlaubte Zeichenketten-Operationen (abschließend!)

Im Abitur darfst du **nur** diese Operationen verwenden. Beispiele mit s = "Drache":

Operation	Java	Beispiel	Ergebnis
Länge	`s.length()`	`s.length()`	6
Zeichen an Position	`s.charAt(i)`	`s.charAt(0)`	'D'
Teilzeichenkette (von a einschließlich bis b ausschließlich)	`s.substring(a, b)`	`s.substring(1, 4)`	"rac"
Verketten	`s + t`	`s + "nei"`	"Drachenei"
Gleichheit	`s.equals(t)`	`s.equals("Drache")`	true
enthält Teilzeichenkette	`s.contains(t)`	`s.contains("ach")`	true
lexikographischer Vergleich	`s.compareTo(t)`	`s.compareTo("Elfe")`	negative Zahl (D vor E)
ASCII-Wert ↔ Zeichen	`(int) c` / `(char) n`	`(int) 'A'` / `(char) 97`	65 / 'a'

Achtung Abi:

- **Nicht** verwenden: `toLowerCase`, `toUpperCase`, `split`, `indexOf`, `trim`, `replace` usw.
- Zeichenketten **immer** mit `equals` vergleichen, nie mit `==`. Die Online-IDE vergleicht mit `==` zwar den Inhalt – echtes Java und das Abitur aber nicht!
- Einzelne Zeichen (`char`) vergleichst du dagegen mit `==`, z. B. `s.charAt(i) == 'a'`.

> 🎯 **Im Lab üben: Level 0.1 „Das Portal: Notation fürs Abitur“ – Punkte gibt es dort automatisch**
>
> Passend zu Teil B und den Aufgaben 4–7: Typnamen übersetzen, Struktogramm-Rätsel, Code-Puzzle und Bug-Hunt – jede Antwort wird sofort geprüft.

Aufgabe 4: Das Rätsel am Portaltor

PAPIER ★

In das Portaltor ist ein Struktogramm eingeritzt. Wer seinen Zweck errät, darf passieren.

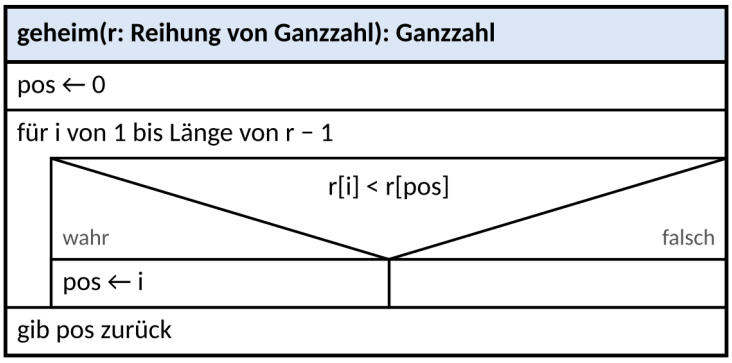


Abbildung 1: Das Struktogramm am Portaltor

a) Gib den Rückgabewert für den Aufruf mit r = {7, 3, 9, 3, 5} an. Notiere dazu die Werte von i und pos. **[2 BE]**

b) Beschreibe, was die Methode allgemein berechnet. Schlage einen passenden Methodennamen vor. **[2 BE]**

c) Erläutere, was bei einer leeren Reihung passiert. Formuliere eine Vorbedingung für r. **[2 BE]**

d) Im Struktogramm wird `<` durch `≤` ersetzt. Gib an, wie sich das Ergebnis für die Reihung aus a) ändert, und begründe. **[2 BE]**

Aufgabe 5: Die Spur der Schatztruhen

PAPIER ★★

Eine Heldin öffnet Schatztruhen der Reihe nach, bis sie genug Gold gesammelt hat.

```
1  public static int schatzSuche(int[] truhe, int ziel) {
2      int summe = 0;
3      int i = 0;   // → Tabellenzeile (Startwerte)
4      while (i < truhe.length && summe < ziel) {
5          summe = summe + truhe[i];
6          i++;   // → Tabellenzeile
7      }
8      return i;
9  }
```

a) Erstelle eine Tracetabelle für den Aufruf `schatzSuche({5, 8, 2, 9, 4}, 15)`: eine Zeile an jeder mit „// → Tabellenzeile“ markierten Stelle. Gib den Rückgabewert an. **[4 BE]**

Schritt	i	summe

Rückgabewert: _____

b) Beschreibe in einem Satz, was die Methode allgemein zurückgibt. **[2 BE]**

c) Gib einen Aufruf an, bei dem die Methode `truhe.length` zurückgibt. Begründe, warum es trotzdem nicht zu einem Zugriff außerhalb der Reihung kommt. **[2 BE]**

Aufgabe 6: Vom Code zum Bauplan

PAPIER ★★

Im Würfelspiel des Dungeons zählt eine 6 als Volltreffer (10 Punkte), eine 1 als Patzer (−5 Punkte). Alle anderen Würfe zählen ihre Augenzahl.

```
public static int wertung(int[] wuerfe) {
    int punkte = 0;
    for (int w : wuerfe) {
        if (w == 6) {
            punkte = punkte + 10;
        } else {
            if (w == 1) {
                punkte = punkte - 5;
            } else {
                punkte = punkte + w;
            }
        }
    }
    return punkte;
}
```

a) Stelle die Methode als Struktogramm nach EH § 5 dar – mit Kopfzeile, Für-jedes-Schleife und Verzweigungen. **[5 BE]**

b) Gib den Rückgabewert für die Würfe {6, 1, 3} und für {2, 6, 6, 1, 5} an. **[2 BE]**

Aufgabe 7: Bug-Hunt im Bauplan

PAPIER ★★

Die Methode soll den Index des **letzten** Vorkommens von x in r liefern, sonst −1. Im Struktogramm steckt **genau ein** Fehler.

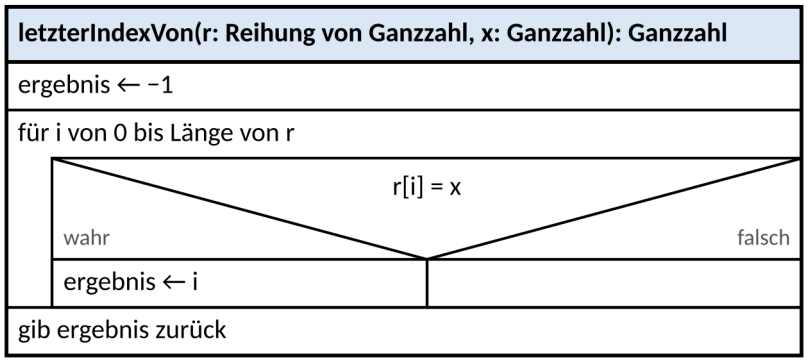


Abbildung 2: Struktogramm mit Fehler

a) Finde den Fehler. Beschreibe, was beim Ausführen mit r = {4, 2, 4, 1} und x = 9 passiert. **[2 BE]**

b) Gib die korrigierte Stelle an. **[1 BE]**

c) Eine Mitschülerin schlägt vor: „Lauf die Reihung von hinten nach vorne durch und gib beim ersten Treffer sofort i zurück.“ Stelle diese Variante als Struktogramm dar (mit Solange-Schleife). Erläutere einen Vorteil. **[4 BE]**

Aufgabe 8: Der Debugger als Trace-Maschine

IDE ★★ Side-Quest

Die Online-IDE kann ein Programm Schritt für Schritt ausführen. So prüfst du deine Tracetabelle aus Aufgabe 5 – und siehst, was im Rechner wirklich passiert.

a) Lege eine neue Datei an. Schreibe eine Klasse `Spur` mit der Methode `schatzSuche` (als `public static`) aus Aufgabe 5. Rufe sie im Hauptprogramm auf:

```
int[] truhe = {5, 8, 2, 9, 4};
println(Spur.schatzSuche(truhe, 15));
```

b) Setze einen **Haltepunkt** (Breakpoint): Klicke links neben die Zeilennummer der Zeile `summe = summe + truhe[i];`. Starte das Programm. Es hält an dieser Zeile an.

c) Gehe mit dem Knopf für den **Einzelschritt** weiter. Beobachte in der **Variablenansicht** die Werte von `i` und `summe`. Vergleiche bei jedem Halt mit deiner Tracetabelle aus Aufgabe 5 und markiere Abweichungen farbig. Wo lagst du falsch – und warum?

d) Baue absichtlich einen Fehler ein: Ändere die Bedingung zu `i <= truhe.length` und rufe die Methode mit ziel = 100 auf. Bei welchem Wert von i bricht das Programm ab? Notiere die Fehlermeldung.

Hinweis: Wie Haltepunkt, Einzelschritt und Variablenansicht genau bedient werden, zeigt dir die Lehrkraft am Beamer.

Für Schnelle

- **Architekten-Probe:** Zeichne ein Struktogramm zu deiner Methode `istPalindrom` aus Level 0 (erst, wenn die Stufe Experte grün ist). Prüfe es mit der Werkzeugkarte „Struktogramm“.
- **Rätselschmiede:** Erfinde selbst ein Struktogramm-Rätsel wie in Aufgabe 4 für ein anderes Team (Lösung auf die Rückseite). Das andere Team löst es und gibt euch Rückmeldung, ob es fehlerfrei und eindeutig lösbar ist.
- **Trace-Training im Lab:** Lab-Level 0.1 enthält weitere Rätsel, Puzzles und Bug-Hunts zur Notation. Zehn Aufgaben in Folge ohne Fehlversuch bringen das Abzeichen „Zehnerserie“.